

06026240 INTELLIGENT SYSTEM DEVELOPMENT

Chapter VIII

8A — เอกสารไม่สะอาด

8B — จากข้อความ สู่คำตอบ

with Dr. Bhattarabhorn Wattanacheep

Lab 8A ต่อยอดจาก Lab 7A โดยเปลี่ยนเงื่อนไขเพียงข้อเดียว

Lab 7A — สิ่งที่ทำไปแล้ว

- ติดตั้ง Ollama และรัน VLM บนเครื่องตัวเอง
- สกัด transcript ออกมาเป็น JSON ตาม schema
- ตรวจสอบความสอดคล้องภายใน
- วัด CER / WER แยกราย attribute
- เอกสารที่ใช้ — สะอาด 100%

Lab 8A — สิ่งที่เปลี่ยนไป

- ใช้ pipeline เดิมทุกอย่าง ไม่แก้โค้ดสักดเลย
- เปลี่ยนแค่ "คุณภาพเอกสารขาเข้า"
- เพิ่มขั้นทำความสะอาดก่อน OCR
- เพิ่มขั้นให้ LLM แก่ข้อความหลัง OCR

คำถามหลัก

LLM กำจัด noise ให้ได้ไหม

1

noise มาจากไหน และเราสร้างมันขึ้นมาเอง ทำไม

คำสั่ง noise

แผนที่ของ noise ในเอกสาร



noise ไม่ได้มีชนิดเดียว และแต่ละชนิดเกิดคนละขั้นตอน

เกิดตอนไหน	ชนิด	ลักษณะที่เห็น	แก้ด้วยอะไร
หมึกบนกระดาษ	ink bleed	ตัวอักษรบวมเบลอ ขอบไม่คม	แก้ยาก ป้องกันที่ต้นทาง
หมึกบนกระดาษ	low ink	เส้นขาดหาย ตัวอักษรจาง	เพิ่มคอนทราสต์
กระดาษ	fold / dirty drum	รอยพับ เส้นแนวตั้งจากลูกกลิ้ง	ลบด้วย morphological
ตอนสแกน / ถ่าย	skew	หน้าเอียงจากการวางกระดาษ	deskew — แก้ได้ดี
ตอนสแกน / ถ่าย	shadow / lighting	ครึ่งหน้ามืด แสงไม่เท่ากัน	background division — แก้ได้ดี
ตอนสแกน / ถ่าย	low resolution	สแกนที่ 100–150 dpi	แก้ไม่ได้ ข้อมูลหายไปแล้ว
ตอนบันทึกไฟล์	JPEG artifact	ขอบตัวอักษรมีคลื่นสีเหลี่ยม	ลดได้บางส่วน

รู้ที่มาก่อน จึงเลือกวิธีแก้ได้ถูก และบางอย่างต้องยอมกลับไปแก้ที่ต้นทางคือสแกนใหม่ ไม่ใช่เขียนโค้ดเพิ่ม

ทำไมต้องสร้าง noise เอง ไม่ไปหาเอกสารสแกนจริงมา

เพราะเราต้องรู้เฉลย 100%

ถ้าเอาเอกสารสแกนจริงมา เราไม่มีทางรู้ว่าค่าจริงในแต่ละช่องคืออะไร นอกจากนั่งพิมพ์เองทุกช่อง
แต่ถ้าเอาเอกสารสะอาดที่มีเฉลยอยู่แล้วจาก Lab 7A มาเติม noise เฉลยเติมยังใช้ได้กับทุกระดับ

หาระดับที่ใช้ในแล็บนี้

ทุกค่าคงที่และผูกกับ seed เดียวกัน ทุกคนจึงได้ภาพเหมือนกันทุกพิกเซล

ระดับ	จำลองสถานการณ์	เอียง	ย่อขนาด	สิ่งที่ใส่เพิ่ม
L0_clean	ต้นฉบับดิจิทัล	0°	100%	ไม่มี — ใช้เป็นเส้นฐาน
L1_light	สแกนเนอร์ดี กระดาษเรียบ	0.4°	100%	ink bleed เบา, JPEG 75–90
L2_watermark	ถ่ายด้วยมือถือแล้วติดลายน้ำ	1.1°	100%	ลายน้ำจำลองตราสถาบัน
L3_titled_copy	ถ่ายซ้ำหลายรอบ	2.3°	100%	ลายน้ำคำว่า COPY
L4_rescan	เครื่องถ่ายเอกสารสำนักงาน	3.8°	100%	L2+L3+ ความมืด

คำสั่ง Gen noise ข้อ 3.4 ในเอกสาร Lab

คำสั่ง sweep เพื่อ Denoise



ในคาบให้รันแบบจำกัดก่อน ครบชุดค่อยรันนอกเวลา

```
# ครบทุกระดับ x ทุกวิธีทำความสะอาด (15 รอบ ใช้เวลานาน)
python3 lab8a_denoise.py sweep \
    -i work/noisy/ -g gt/transcript_gt.json -o work/out/

# ในคาบเรียน – สองระดับ วิธีเดียว
python3 lab8a_denoise.py sweep \
    -i work/noisy/ -g gt/transcript_gt.json -o work/out/ \
    --levels L0_clean,L3_degraded --methods none
```

จุดพัง (breaking point) คือระดับที่ accuracy เริ่มร่วงอย่างชัดเจน มันมีค่ามากกว่าค่า accuracy ที่ระดับใดระดับหนึ่ง เพราะบอกเราว่าควรตั้งเกณฑ์คุณภาพเอกสารเข้าไว้ที่เท่าไร

3

ทำความสะอาดภาพด้วย OpenCV

ลำดับสี่ขั้น และเหตุผลของแต่ละขั้น

ลำดับนี้เปลี่ยนไม่ได้ เพราะแต่ละขั้นสมมติว่าขั้นก่อนหน้าทำเสร็จแล้ว

1

แก้ภาพเอียง

deskew

ต้องมาก่อนเสมอ เพราะขั้นถัดไปทั้งหมดสมมติว่าบรรทัดข้อความอยู่ในแนวนอน ถ้าภาพยังเอียง การประมาณพื้นหลังจะเพี้ยนตามไปด้วย

2

ลบเงา

background division

ประมาณระดับแสงพื้นหลังด้วย morphological closing ที่ใช้ kernel ใหญ่กว่าตัวอักษรมาก แล้วเอาภาพเดิมหารด้วยพื้นหลังนั้น

3

ลด noise

fastNlMeansDenoising

ไม่ใช่ GaussianBlur เพราะ Gaussian จะเบลอขอบตัวอักษรไปด้วย ซึ่งเป็นสิ่งเดียวที่เราต้องรักษาไว้

4

binarize

adaptiveThreshold (เฉพาะ heavy)

ไม่ใช่ Otsu ทั้งภาพ เพราะ Otsu ใช้ค่าตัดค่าเดียวทั้งหน้า พอเจอหน้าที่ครึ่งหนึ่งมืดกว่าจะได้ครึ่งหน้าดำสนิท

ประเด็นที่นักศึกษาส่วนใหญ่จะเดาผิด

"สะอาดขึ้นในสายตาคน" ไม่ได้แปลว่า OCR อ่านได้ดีขึ้น

Typhoon-OCR ฝึกมาบนเอกสารสแกนจริงจำนวนมหาศาล จึงคุ้นเคยกับ noise ระดับหนึ่งอยู่แล้ว การ binarize แรง ๆ ตัดข้อมูลระดับสีเทาทิ้ง ซึ่งเป็นข้อมูลที่โมเดลใช้แยกสระและวรรณยุกต์ไทย

4

ให้ LLM แก่ข้อความ และวัดว่ามันทำอะไรลงไป

สมมติเราให้ LLM แก่ข้อความ แล้ว accuracy เพิ่มจาก 0.712 เป็น 0.719

ก่อนแก้	ถูก 712 ช่อง	ผิด 288 ช่อง	
หลังแก้	ถูก 719 ช่อง	ผิด 281 ช่อง	<- ดีขึ้น 7 ช่อง

แต่ถ้าแยกดูว่าเกิดอะไรขึ้นจริง ๆ

แก้ผิดให้เป็นถูก	15 ช่อง	<- ช่องที่ดีขึ้น
แก้ถูกให้เป็นผิด	8 ช่อง	<- เดิมถูก แต่พังเพราะการแก้ (และไม่รู้ว่าช่องไหน)

สุทธิ +7 ช่อง

หลักการที่ใช้ได้กับงาน LLM ทุกประเภท

prompt คือการขอร้อง โค้ดคือการบังคับ

prompt ที่เขียนดีจะลดโอกาสผิดพลาดได้มาก แต่ไม่เคยลดเหลือศูนย์
อะไรที่ต้องถูกเสมอ ต้องบังคับด้วยโค้ด ไม่ใช่ฝากไว้กับ prompt

เรารู้ล่วงหน้าว่าค่าที่ถูกต้องมีได้กี่แบบ อะไรที่อยู่นอกชุดนั้นคืออ่านผิดแน่นอน

ฟังก์ชัน	ข้อจำกัดที่บังคับ	เมื่อถามจะทำอย่างไร
snap_grade	เกรดต้องอยู่ในชุด A, B+, B, C+, C, D+, D, F, W, S, U, I, P, T	ถ้าใกล้เคียงหลายตัว คืน null ให้คนตรวจ
snap_code	รหัสต้องเป็นเลข 8 หลัก และมีอยู่ในทะเบียนรายวิชา	ต่างหลักเดียว → แก้ให้ ต่างสองหลักขึ้นไป → คืน null
snap_credits	หน่วยกิตต้องเป็นจำนวนเต็ม 0-12	นอกช่วง → คืน null

งานที่ต้องส่ง (งานเดี่ยวส่งใน discord กลุ่มตัวเอง)



เลือกมา 1 ภาพ ส่งก่อนคาบถัดไป

ไฟล์	รายละเอียด
work/noisy/manifest.json (ใน code จะ save ไว้ path นี้ ส่งแต่ไฟล์ json ไม่ต้องเอา folder ครบ)	พารามิเตอร์ที่ใช้สร้าง noise
compare.png	ภาพเปรียบเทียบ 5 ระดับ พร้อมป้ายกำกับ
curve.png	กราฟ accuracy เทียบระดับ noise แยกเส้นตามวิธี

ภาพมี noise



OpenCV ทำความสะอาดภาพ

← ลบลายน้ำ แก้เอียง ลด noise



Typhoon OCR อ่านภาพ

← ทน noise ได้ระดับหนึ่ง



Qwen แปลง Markdown เป็น JSON

← จัดโครงสร้าง ไม่ได้ลบ noise



Qwen post-correction (คำสั่ง fix)

← พยายามแก้คำ OCR ผิด



เทียบ Ground Truth

Lab 8B — จากข้อความ สู่คำตอบ

สกัดเป็น JSON · บันทึกลงฐานข้อมูล · ตอบคำถามภาษาไทย

คำถามแรกที่คุณควรถาม

ทำไมไม่โยนข้อความทั้งก้อน ให้ LLM แล้วถามตรง ๆ เลย

คำถามนี้ถูกต้องและควรถาม คำตอบคือทำได้จริง และได้ผลดีพอควรกับคำถามบางประเภท
แต่ฟังกับคำถามที่นักศึกษาถามจริง ๆ

ถาม LLM ตรง ๆ ได้ผลกับคำถามแบบไหน



สังเกตว่าสามแถวล่างคือคำถามที่นักศึกษาถามจริงเกือบทั้งหมด

ชนิดคำถาม	ตัวอย่าง	ถาม LLM กับข้อความตรง ๆ	ผ่านฐานข้อมูล
ค้นหาข้อความ	วิชา 06026240 ชื่ออะไร	ทำได้ดี	ทำได้ดี
สรุปความ	หลักสูตรนี้เน้นอะไร	ทำได้ดี	ทำไม่ได้
คำนวณ	ปี 3 เทอม 1 กี่หน่วยกิต	นับผิดบ่อยมาก	ถูกทุกครั้ง
ความสัมพันธ์	วิชาไหนต้องเรียน X มาก่อน	ตกหล่นเมื่อข้อมูลอยู่คนละหน้า	ถูกทุกครั้ง
เปรียบเทียบ	เทอมไหนหนักที่สุด	เดาเอา	ถูกทุกครั้ง

สิ่งที่ LLM เก่ง กับสิ่งที่ LLM ไม่เก่ง



KMITL
สถาบันเทคโนโลยี
พระจอมเกล้า
เจ้าพระยา

KMITL
FIGHT

ทางออกคือให้แต่ละฝ่ายทำสิ่งที่ตัวเองเก่ง

เก่งมาก — แปลภาษาคนเป็นภาษาเครื่อง

- เปลี่ยนคำถามภาษาไทยให้เป็นคำสั่ง SQL
- เข้าใจคำถามที่ถามด้วยภาษาพูด ไม่ตรงชื่อคอลัมน์
- สรุปผลลัพธ์ที่เป็นตัวเลขให้เป็นประโยคภาษาไทย

ไม่เก่ง — เป็นเครื่องคิดเลข

- บวกเลข 8 ตัวที่กระจายอยู่ในข้อความ 3 หน้า
- ไล่ความสัมพันธ์ข้ามหน้าโดยไม่ตกหล่น
- ที่ร้ายกว่านั้นคือ มันคิดแบบที่ดูน่าเชื่อถือ

แต่ละก้าวเพิ่มความสามารถอย่างหนึ่งที่ก้าวก่อนหน้านี้ไม่มี

Lab 7B	Lab 8B / 2	Lab 8B / 3	Lab 8B / 5
+-----+	+-----+	+-----+	+-----+
Markdown -->	JSON -->	SQLite ->	Answer
+-----+	+-----+	+-----+	+-----+
	Pydantic + repair loop	constraint + check x 7	NL -> SQL + guardrail

1

ออกแบบ Schema ก่อน แล้วค่อยสกัด

ถูก ออกแบบ schema -> สกัดตาม schema -> ตรวจสอบด้วย schema
ผิด สกัดก่อน -> ดูว่าได้อะไรมา -> ค่อยคิด schema ที่หลัง

1. แต่ละหน้าได้ชื่อฟิลด์ไม่ตรงกัน หน้าหนึ่ง "หน่วยกิต" อีกหน้า "credit" พอจะรวมข้อมูลทั้งเล่มก็รวมไม่ได้
2. ไม่มีเกณฑ์ตัดสินว่า "ผิด" คืออะไร จึงตรวจสอบอัตโนมัติไม่ได้เลย ต้องนั่งอ่านเองทุกบรรทัด

courses กับ plan แยกกัน ทั้งที่ดูเหมือนซ้ำซ้อน — นี่คือการ normalize

ส่วน	เก็บอะไร	มาจากหมวดไหนของเล่ม
program	ชื่อหลักสูตร ปริญญา หน่วยกิตรวม จำนวนปี	หน้าแรกและหมวดโครงสร้างหลักสูตร
courses	รหัส ชื่อ หน่วยกิต ชั่วโมง ท-ป-อ คำอธิบาย	หมวดคำอธิบายรายวิชา
plan	ปี เทอม รหัสวิชา หน่วยกิต	หมวดแผนการศึกษา
prerequisites	วิชาบังคับก่อน / วิชาเรียนควบ	ท้ายคำอธิบายรายวิชาแต่ละวิชา

alt_group — วิชาเลือกอย่างไรอย่างหนึ่ง



เล่นเขียนว่า "06026259 หรือ 06026260" เราแตกเป็นสองแถวที่มี alt_group เดียวกัน

year	semester	code	credits	alt_group
2	1	06026240	3	(ว่าง)
2	1	06026241	3	(ว่าง)
2	1	06026259	3	elect_y2s1
2	1	06026260	3	elect_y2s1

2

สัปดาห์ JSON พร้อมวงจรซ่อม

วงจรซ่อม (repair loop)

LLM ที่ถูกสั่งให้ตอบเป็น JSON จะตอบผิดรูปแบบเป็นบางครั้ง ไม่ว่าเขียน prompt ดีแค่ไหน

```
สกัด --> ตรวจสอบด้วย Pydantic --> ผ่าน --> บันทึก
^
|
|   ไม่ผ่าน
|   v
+---- ส่งรายการข้อผิดพลาดกลับให้แก้ (ไม่เกิน 3 รอบ)
      |
      |   ครบ 3 รอบแล้วยังไม่ผ่าน
      |   v
      |   ทำเครื่องหมายให้คนตรวจ
```

วิธีรับมือที่ได้ผลคือ ไม่หวังให้มันถูกต้องแต่รอบแรก แต่สร้างวงจรที่บอกมันว่าผิดตรงไหน แล้วให้แก้

lab8b_run/.....conversion.json

ข้อความ error ต้องบอกตำแหน่งให้ชัด



จุดที่ทำให้ repair loop ได้ผลหรือไม่ได้ผล อยู่ที่คุณภาพของข้อความ error

บอกแค่ว่าผิด

- "รหัสวิชาผิดรูปแบบ"
- โมเดลไม่รู้ว่าจะต้องแก้รายการไหน
- มักรู้ทั้งก่อนใหม่
- ทำให้ส่วนที่ถูกอยู่แล้วพังไปด้วย

บอกตำแหน่งชัด

- "ตำแหน่ง plan -> 12 -> code:
- รหัสวิชาต้องเป็นตัวเลข 8 หลัก แต่ได้ 0602624"
- โมเดลแก้เฉพาะจุดนั้น
- ส่วนอื่นไม่ถูกแตะ

สิ่งที่ต้องระวังที่สุดในส่วนนี้

validation ที่ผ่าน ไม่ได้แปลว่าข้อมูลถูก

Pydantic ตรวจได้แค่ว่า "หน้าตาถูกรูปแบบไหม" ไม่ได้ตรวจว่า "เป็นความจริงไหม"
รหัสวิชา 06026999 ผ่าน Pydantic สบายมาก ทั้งที่ไม่มีวิชานี้อยู่จริงในเล่ม

3

กฎตรวจสอบความสอดคล้อง 7 ข้อ

สามขั้นตอนของการตรวจ — Pydantic ตรวจสอบแต่ละขั้น · constraint ตรวจสอบความสัมพันธ์ · กฎเจ็ดข้อตรวจทั้งหมด

รหัส	กฎ	จับความผิดแบบไหน
CHK1	หน่วยกิตรวมของแผน = หน่วยกิตที่ประกาศ	สังกัดแผนตกหล่นไปทั้งภาคเรียน
CHK2	ทุกรหัสในแผน มีคำอธิบายรายวิชา	สังกัดรหัสผิดหลัก หรือหมวดคำอธิบายไม่ครบ
CHK3	รหัสวิชาเป็นตัวเลข 8 หลัก	OCR อ่านตัวอักษรปนเข้ามาในรหัส
CHK4	หน่วยกิตในแผน = หน่วยกิตในคำอธิบาย	สังกัดตัวเลขผิดในแหล่งใดแหล่งหนึ่ง
CHK5	วิชาบังคับก่อน อยู่ภาคที่มาก่อนจริง	จับคู่ปี/เทอมผิด หรือเล่มเขียนผิดเอง
CHK6	ไม่มีวิชาซ้ำในภาคเรียนเดียวกัน	สังกัดตารางซ้ำสองรอบ
CHK7	หน่วยกิตต่อภาคอยู่ระหว่าง 9–22	ตกหล่นบางวิชา หรือเกินมาจากหน้าอื่น

กฎที่เดือนฝอย แยกว่าไม่มีกฎเลย

ข้อยกเว้น	พบที่ไหน	กฎที่ต้องรู้จัก
วิชาเลือกอย่างใดอย่างหนึ่ง	ทุกหลักสูตร	CHK1 CHK6 CHK7 — นับ alt_group ครั้งเดียว
ภาคสหกิจศึกษา / ฝึกงาน	หลักสูตรที่มีสหกิจ	CHK7 — ภาคนั้นมีวิชาเดียว 6 หน่วยกิต
ภาคฤดูร้อน	หลายหลักสูตร	CHK7 — หน่วยกิตน้อยเป็นเรื่องปกติ
วิชาเรียนควบ (co)	วิชาบรรยายคู่ปฏิบัติการ	CHK5 — อยู่ภาคเดียวกันได้ตามระเบียบ

ไม่มีกฎใดได้ทั้งสองอย่าง — สิ่งที่ทำได้คือรู้ว่าจุดบอดอยู่ตรงไหน

ตัวอย่างจุดบอดจริงในโค้ดนี้

- CHK7 ยกเว้นภาคที่มีวิชา 6 หน่วยกิตขึ้นไป
- ถ้าสังกัดหน่วยกิตผิดจาก 3 เป็น 6
- ภาคเรียนนั้นจะถูกยกเว้นทันที
- CHK7 จะเจียบทั้งที่ข้อมูลผิด

วิธีรับมือ

- กฎที่เข้มขึ้น จับได้มากขึ้น แต่เดือนผิดมากขึ้น
- กฎที่ผ่อนลง เดือนผิดน้อยลง แต่ปล่อยหลุดมากขึ้น
- บันทึกจุดบอดไว้ในโค้ดตรง ๆ อย่าปล่อยผ่าน
- วางกฎอีกข้อไปคุมตรงนั้น — ในกรณีนี้คือ CHK4

เมื่อกฎเตือน ต้องแยกให้ออกก่อนแก้



สองกรณีนี้แก้คนละที่โดยสิ้นเชิง

(ก) สกัดผิด -> กลับไปแก้ prompt หรือแก้ JSON

(ข) เล่มเขียนแบบนั้นจริง -> ต้องแก้กฎให้รู้จักข้อยกเว้นนี้

การแยกสองกรณีนี้ต้องเปิดเล่มหลักสูตรหน้านั้นดูด้วยตัวเอง ไม่มีทางลัด และเป็นทักษะที่สำคัญที่สุดอย่างหนึ่งของงานสกัดข้อมูล

4

ถามเป็นภาษาคน ตอบด้วย SQL

คำถามภาษาไทย

- (1) LLM สร้าง SQL <- โมเดลทำสิ่งที่ตัวเองเก่ง
- (2) guard_sql ตรวจสอบ <- ด้านชั้นที่ 2 ปฏิเสธคำสั่งอันตราย เติม LIMIT
- (3) ฐานข้อมูลรัน <- ด้านชั้นที่ 1 เปิดแบบอ่านอย่างเดียว
- (4) LLM สรุปเป็นภาษาไทย <- ใช้ได้เฉพาะตัวเลขที่ได้จากชั้น 3

ทำไมต้องสองชั้น — ชั้นอ่านอย่างเดียวกับการแก้ไขข้อมูลได้ แต่กับการดึงข้อมูลจนล้นหรือ query ที่รันไม่จบไม่ได้ หลักการทั่วไปคือ อย่าให้มีจุดเดียวที่พังแล้วพังหมด

สิ่งที่ระบบต้องทำได้ และหลายระบบทำไม่ได้

ระบบที่ตอบทุกคำถามได้เสมอ คือระบบที่แต่งคำตอบเมื่อไม่มีข้อมูล

เมื่อ SQL รันได้แต่ไม่มีแถวกลับมา ระบบตอบทันทีว่า "ไม่พบข้อมูลนี้ในเล่มหลักสูตร" โดยไม่ส่งต่อให้ LLM สรุป
ชุดคำถามทองจึงต้องมีอย่างน้อยสองข้อที่คำตอบที่ถูกคือ "ไม่รู้"

ประเมินผล — สองตัวเลขที่ต้องแยกกันเสมอ



KMITL
สถาบันเทคโนโลยี
พระจอมเกล้า
เจ้าพระยา

KMITL
FIGHT

ถ้าดูตัวเลขรวมตัวเดียว จะแยกไม่ออกว่าปัญหาอยู่ที่ไหน

94%

SQL รันผ่าน
โมเดลเขียน SQL ที่ใช้งานได้ก็ข้อ
ถ้าด้า → แก้ที่ schema หรือเพิ่ม VIEW

81%

ตอบถูก
จากข้อที่รันผ่าน ตอบตรงก็ข้อ
ถ้าด้า → แก้ที่ prompt หรือเพิ่มตัวอย่าง

ให้คะแนนที่ "ผลลัพธ์ของ SQL" ไม่ใช่ที่ "ข้อความคำตอบ" — เพราะ "9 หน่วยกิต" กับ "รวมทั้งสิ้น 9 หน่วยกิต" ควรได้คะแนนเท่ากัน แต่การเทียบข้อความตรง ๆ จะนับเป็นผิด

งานที่ต้องส่ง (งานเดี่ยวส่งใน discord กลุ่มตัวเอง)



เลือกมา 1 หลักสูตร ส่งก่อนคาบถัดไป

ไฟล์	รายละเอียด
schema/curriculum.schema.json	JSON Schema ที่ออกแบบเอง (แก้จากของเดิมได้)
schema/schema.sql	คำสั่งสร้างตารางและ VIEW
curriculum.json + .conversion.json	ผลการสเก็ต พร้อมบันทึกว่าซ่อมกี่รอบ
curriculum.db	ฐานข้อมูลที่โหลดเสร็จแล้ว
verify.json	ผลการตรวจ 7 ข้อ พร้อมแยกกรณี (ก)/(ข)
gold_questions.json	ชุดคำถาม 30 ข้อ ครบตามเงื่อนไข
eval_result.json	ผลการประเมิน แยก SQL รันผ่าน กับ ตอบถูก

วางไว้โฟลเดอร์หลัก ocr_system ตำแหน่งเดียวกับ requirements.txt

ใน Terminal ของ VS Code:

เปิด env ก่อน

Windows

.venv\Scripts\activate

Mac/Linux

source .venv/bin/activate

แล้วทุกระบบรันเหมือนกัน:

python run_lab8b.py

ถ้ามีผล Lab 7B แล้ว:

python run_lab8b.py --skip-lab7

ผลทั้งหมดจะอยู่ใน:

work/lab8b_run/

Lab 7B

ภาพเอกสาร



Typhoon OCR



Markdown

Qwen Extractor



JSON รายวิชา

ประเมินเทียบ Ground Truth

Lab 8B

JSON จาก Lab 7B



แปลง Schema

SQLite Database



คำถามภาษาไทย



Qwen สร้าง SQL



SQLite ประมวลผล



Qwen สรุปคำตอบภาษาไทย



ประเมินเทียบคำตอบทอง